Fewer points. Same 3D scene.

A C++20 point-cloud video engine with an optional CUDA backend and measured correctness.



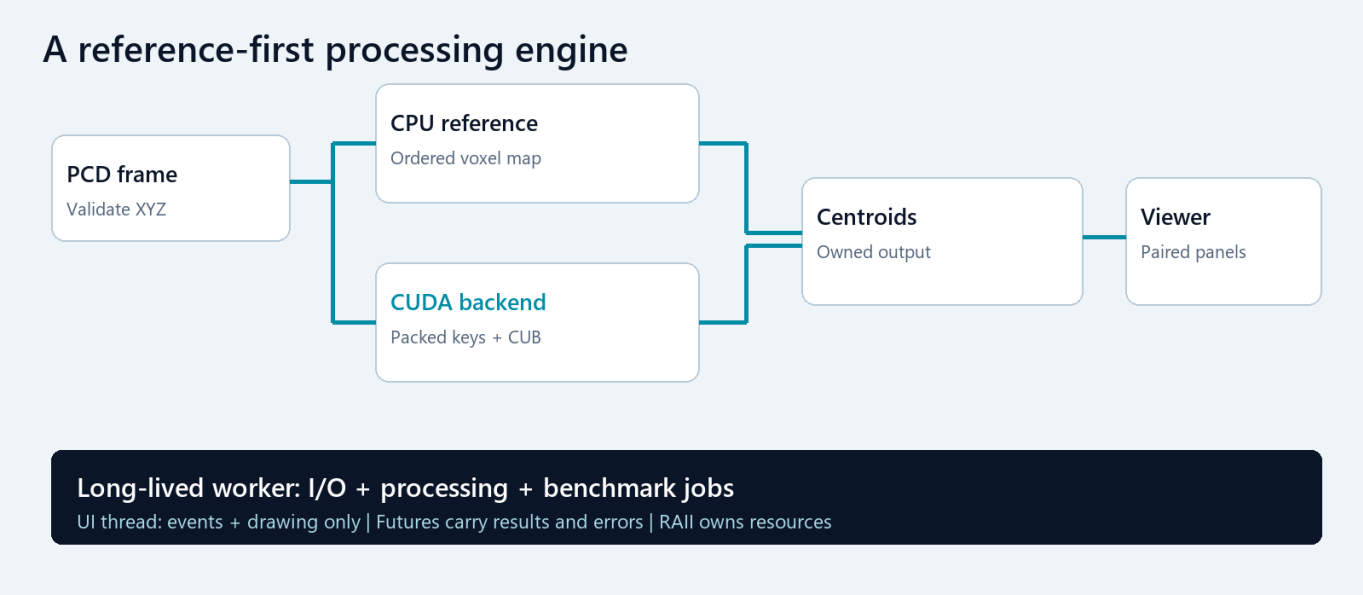
Actual app capture: recorded Oxford Spires frames, original at left and CUDA centroids at right. Capture is from WSL; a separate native Windows CPU build was also tested.

MEASURED SESSION	RESULT
First frame, voxel 0.3	55,518 input points -> 9,943 centroids; 82.09% reduction
900 paired timing samples	CPU median 5.343 ms; CUDA median 1.205 ms
Ratio of pooled medians	4.43x observed in this WSL session
Validation	35 native Windows CPU tests; 41 WSL CUDA-build tests passed

The project demonstrates deterministic reference behavior, optional acceleration, explicit ownership, responsive desktop processing and reproducible test/build commands. It is a recorded-data processing application, not a physical simulator.

A small engine with clear boundaries

I/O, processing, benchmarking and visualization are independent responsibilities.



Both backends return owned host-side point clouds. The viewer never runs OpenGL calls on the processing worker.

COMPONENT	RESPONSIBILITY
PCD reader	Reads uncompressed little-endian binary PCD; derives XYZ offsets from header fields and padding.
CPU reference	Maps signed integer voxel keys to a weighted centroid accumulator; outputs sorted keys.
CUDA backend	Generates keys, radix-sorts, reduces by key, emits centroids and copies results to host.
Worker + futures	Serializes I/O and processing; propagates errors; keeps the CUDA workspace alive across jobs.
GLFW/OpenGL viewer	Shows matched original/simplified frames, synchronized cameras, detail cells and playback.

Ownership: vectors own clouds; RAII buffers own device allocations; a long-lived worker owns its thread-local CUDA workspace. Closing waits for active work. CPU-only builds do not enable the CUDA language or require its toolkit.

One occupied voxel, one centroid

Voxel size controls sampling density while preserving the broad shape of a scan.

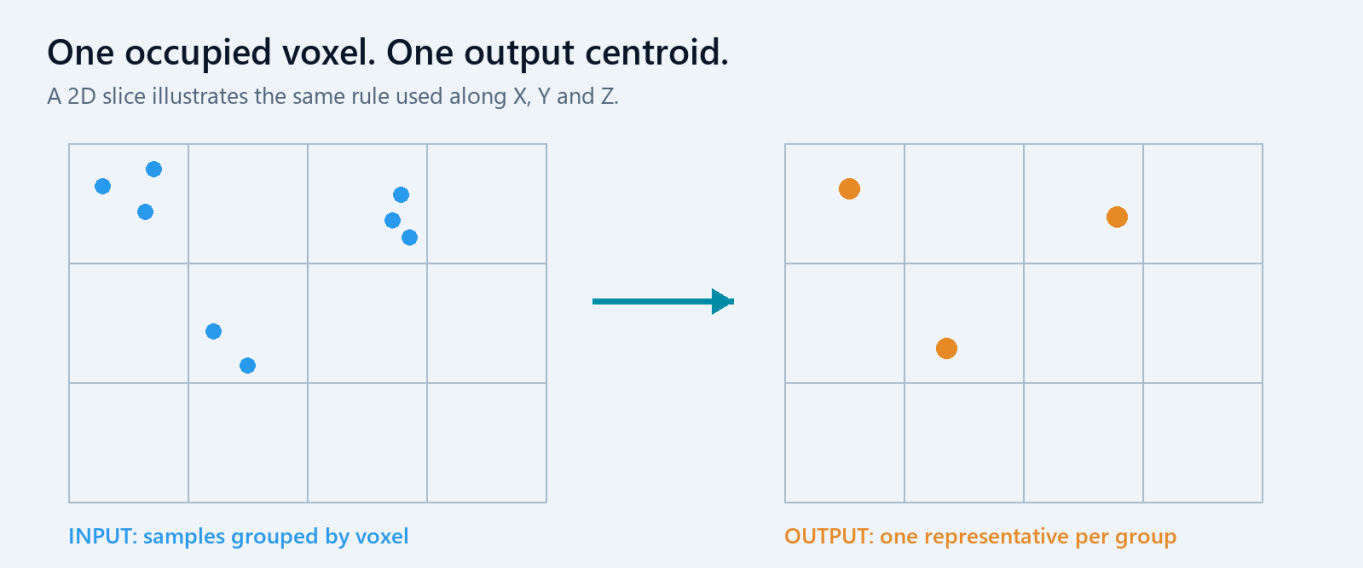


Illustration: a 2D slice of a 3D rule. Blue samples in each occupied cell become one orange centroid. These dots are explanatory, not benchmark data.

```
key = (floor(x / s), floor(y / s), floor(z / s))
centroid = arithmetic mean of all points sharing that key
```

For s = 1, points (0.1, 0.2, 0.3) and (0.2, 0.3, 0.4) share key (0, 0, 0). Their representative is (0.15, 0.25, 0.35). A point at x = -0.1 belongs to x-key -1, not 0: floor is essential for negative coordinates.

Boundary and numeric rules

Cells are half-open intervals [k*s, (k+1)*s) in each axis. Exact boundary points move into the next cell. Empty input produces empty output; duplicates collapse into one representative. Nonfinite coordinates, invalid voxel sizes and out-of-range integer keys are rejected.

The CPU implementation uses a running weighted mean rather than accumulating a potentially overflowing coordinate sum. Sorted map keys make repeated execution deterministic for the same input order. Floating-point centroid values are not promised to be invariant to arbitrary input permutations.

Acceleration without changing the contract

The GPU accelerates grouping and centroid reduction; it does not alter the required voxel partition.

STAGE	CUDA IMPLEMENTATION
1. Validate	Host checks finite coordinates, voxel size and signed key range.
2. Generate	A fused kernel creates collision-free packed uint64 keys and weighted aggregates.
3. Sort	CUB DeviceRadixSort groups equal keys into contiguous runs.
4. Reduce	CUB DeviceReduce::ReduceByKey combines centroids and point counts.
5. Emit	A kernel emits centroid points; a completed device-to-host copy returns owned output.

Why the fast path is safe

Mixed-radix packing is used only when the product of the observed key extents fits uint64. Otherwise, a full signed triple-key Thrust sort/reduce fallback preserves correct grouping. The code avoids treating a lossy hash as a unique voxel identifier.

Workspace reuse and floating-point comparison

Growable RAII device buffers and CUB scratch space persist on the worker thread. Capacity grows when required; it is not reallocated on every warmed frame. Backend calls still include required growth allocation, validation, transfers and synchronization.

CPU and GPU reduction orders differ, so bitwise equality is the wrong acceptance test. The project compares sorted outputs, counts and finite coordinates with absolute and relative tolerances of 1e-10. The new 900-sample run passed every comparison; maximum observed coordinate error was 1.78e-15.

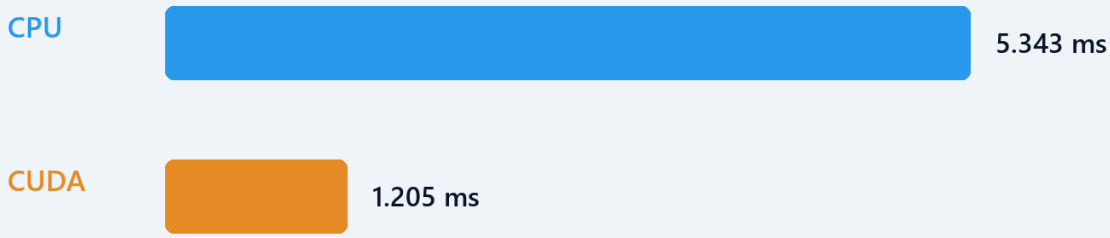
This is tested numerical agreement for the exercised inputs, not a proof for every finite floating-point dataset. Full-range fallback, repeated workspace growth and large signed offsets have dedicated GPU tests.

Measure the completed operation

Fresh session: first 30 timestamp-ordered frames, 3 warm-ups and 30 repetitions per backend per frame.

Measured processing latency

30 frames x 30 repetitions | 900 paired samples | voxel size 0.1



4.43x observed ratio of medians

Full completed backend call; file I/O and rendering excluded.

One WSL session on Ryzen 7 7735HS + RTX 4050 Laptop. Not a universal speedup.

BACKEND	P10 (ms)	MEDIAN (ms)	P90 (ms)
CPU	4.933	5.343	7.684
CUDA	0.966	1.205	1.762

The pooled-median ratio is 4.43x; the ratio of summed measured CPU and CUDA time is 4.49x. These are different summaries of the same work. All 900 paired comparisons passed. P10-P90 describes sample variation across both repetitions and frames, not a confidence interval.

Included: validation, required allocation, transfers and completed GPU output. **Excluded:** file loading, rendering, warm-ups, correctness checking and CSV writing. CPU/CUDA measured order alternates. No app capture or compilation was launched concurrently by the authoring workflow during this run.

Hardware: Ryzen 7 7735HS; RTX 4050 Laptop, 6,141 MiB; CUDA 13.2.86; driver 595.79; Ubuntu/WSL Release. CPU is the single-threaded ordered-map reference; no parallel CPU baseline was tested. Power, clocks, thermals and OS activity were not controlled. Results are observations, not universal speedups. Raw evidence: output/evidence/benchmark-session.csv.

Inspect geometry. Replay recorded frames.

The original and simplified panels always represent the same completed frame.



Native Windows CPU capture, voxel 0.3: local detail shows actual occupied-cell wireframes and their centroid representatives. The crop is explicitly separate from global point counts.

The viewer supports drag/drop and a cached background file browser, largest-file-first listing, orbit/pan/zoom, a fixed local voxel region, and configurable voxel size. A single loaded file offers Play Clip, which discovers neighboring recorded frames without requiring another file selection.

A sequence runs one selected-backend call per frame. It does not repeat benchmark work in the playback loop. Frames are timestamp-ordered; separate clip folders play consecutively. Pause, previous/next, restart, loop and target-FPS controls live in one playback bar.

A second resizable window keeps benchmark settings and results together. It uses the main app input, recalculates on every run, saves distinct CSVs automatically, exposes median latency and P10-P90 graphs, and provides precise hover values. Main-screen metrics update after completion.

Playback uses an adjustable fixed rate, not exact source timestamp intervals. Actual playback FPS includes the effect of I/O and rendering and differs from reciprocal processing latency. Camera framing stays fixed within a clip; no artificial motion is added to stationary recordings.

Windows: CPU verified. CUDA scoped honestly.

The Linux executable and native Windows executable are distinct builds of the same source.

TARGET	RESULT / EVIDENCE
Windows native CPU	MSVC 19.37, Visual Studio 2022, x64 Release. Build succeeded; all 35 CPU/loader/workflow tests passed.
Windows native GUI	GLFW Win32 + OpenGL. Actual renderer: AMD Radeon Graphics. 60 real PCD frames loaded, downsampled and captured.
Windows native CUDA	Configure attempted and failed: "No CUDA toolset found." No native Windows CUDA Toolkit/toolset was detected. GPU support is NOT verified here.
WSL CUDA	Release build, all 41 tests passed (35 shared + 6 GPU). 60 recorded frames captured using CUDA. Drawing used llvmpipe; compute used NVIDIA CUDA.
Cross-build sanity	Native CLI read 55,518 points and emitted 20,345 centroids at voxel 0.1, matching the established WSL reference counts.

What changed for native builds

An opt-in dependency-fetch option obtains GLFW 3.4 and GoogleTest 1.17.0 into the build tree when installed packages are absent. MSVC warnings found during qualification were corrected. No CUDA Toolkit or system-wide development package was installed on the borrowed Windows machine.

Dependencies and media tooling used for this task reside in project build folders. Tests construct their own small fixtures. A failing Windows CUDA configure is recorded as a blocker, not reported as a successful GPU test. The executable requires the appropriate Microsoft VC++ runtime on other computers.

A repeatable starting point

Build commands, evidence and remaining limits are part of the deliverable.

```
# Native Windows, from a terminal with CMake and Git
cmake -S . -B build/windows-cpu -G "Visual Studio 17 2022" -A x64
  -DCLOUDFORGE_BUILD_VIEWER=ON -DCLOUDFORGE_FETCH_DEPS=ON
  -DBUILD_TESTING=ON -DCLOUDFORGE_ENABLE_CUDA=OFF
cmake --build build/windows-cpu --config Release
ctest --test-dir build/windows-cpu -C Release --output-on-failure
```

```
# Ubuntu / WSL (CUDA compiler path may be needed at configure)
cmake --preset cuda-release
cmake --build --preset cuda-release
ctest --preset cuda-release
bash scripts/run-demo.sh
```

Commands spanning multiple printed lines must be entered as one command or use the continuation syntax of the chosen shell. Windows package: output/windows/. Tests and CUDA configure failure logs: output/evidence/. Media scripts and the raw timing CSV are retained for review.

Limits and next experiments

The PCD reader currently accepts uncompressed little-endian binary scalar float32 XYZ; extra attributes are skipped. Playback has no timestamp-accurate scheduling, bounded prefetch or cancellation inside a running kernel. CUDA crossover depends on cloud size and key distribution. Next: workload scaling, per-stage profiling, bounded prefetch and native Windows CUDA qualification once a compatible toolkit is installed.

Sources and visual attribution

Project implementation, tests and raw CSVs are primary evidence. External references: Oxford Spires Dataset - Tao et al.; NVIDIA CUB API; GLFW; GoogleTest.

Screenshots/footage are original app captures visualizing Oxford Spires data. Dataset authors: Yifu Tao, Miguel Angel Munoz-Banon, Lintong Zhang, Jiahao Wang, Lanke Frank Tarimo Fu and Maurice Fallon. Dataset-derived visuals and these media documents are shared under CC BY-NC-SA 4.0; the project code retains its separate MIT license. No raw dataset is bundled.